6

Input Space Partitioning

Engineers take ideas invented by quick thinkers and build products for slow thinkers.

In a very fundamental way, all testing is about choosing elements from the input space of the software being tested. Input space partitioning takes the view that we can directly divide the input space according to logical partitionings of the inputs. The four chapters in Part II are based on the four structures defined in Chapter 5 and are ordered to reflect the RIPR model of Chapter 2. Input space partitioning teaches test design in a way that is independent of the RIPR model–we only use the input space of the software under test. The next chapter is on graphs, and the criteria ensure reachability. Using logic expressions to generate tests (Chapter 8) ensures infection, and mutation analysis (Chapter 9) ensures propagation.

The *input domain* is defined in terms of the possible values that the input parameters can have. The input parameters can be method parameters and non-local variables (in unit testing), objects representing current state (in class or integration testing), or user-level inputs to a program (in system testing), depending on what kind of software artifact is being analyzed. The input domain is then partitioned into regions that are assumed to contain equally useful values from a testing perspective, and values are selected from each region.

This way of testing has several advantages. It is fairly easy to get started because it can be applied with no automation and very little training. The tester does not need to understand the implementation; everything is based on a description of the inputs. It is also simple to "tune" the technique to get more or fewer tests.

Consider an abstract partition q over some domain D. The partition q defines a set of equivalence classes, which we simply call *blocks*, B_q[1].

Together the blocks are complete, that is they do not miss any elements of *D*:

$$\bigcup_{b \in B_q} b = D$$

and the blocks are pairwise disjoint, that is no element of *D* is in more than one block :

$$b_i \cap b_j = \emptyset,\ i \neq j;\ b_i, b_j \in B_q$$

This is illustrated in Figure 6.1. The input domain *D* is partitioned into three blocks, b_1, b_2, and b_3. The partition defines the values contained in each block and is usually designed using knowledge of what the software is supposed to do.

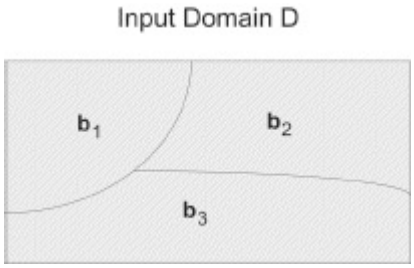

Figure 6.1. Partitioning of input domain *D* into three blocks.

The underlying assumption of partition coverage is that any test in a block is as good as any other for testing. Several partitions are sometimes considered together, which, if not done carefully, leads to a combinatorial explosion of test cases.

A common way to apply input space partitioning is to start by considering the domain of each parameter separately, partitioning each domain's possible values into blocks, and then combining the blocks for each parameter. Sometimes the parameters are considered completely independently, and sometimes they are considered in conjunction with each other, usually by taking the semantics of the program into account. This process is called *input domain modeling* and is discussed in the next section.

Each partition is usually based on some *characteristic C* of the program, the program's inputs, or the program's environment. Some possible characteristic examples are:

- Input *X* is null

- Order of file *F* (sorted, inverse sorted, arbitrary)
- Min separation distance of two aircraft
- Input device (DVD, CD, VCR, computer, …)

Each characteristic *C* allows the tester to define a partition. Formally, a partition must satisfy the two properties identified earlier:

1. The partition must cover the entire domain (completeness)
2. The blocks must not overlap (disjoint)

As an example, consider the characteristic "order of file *F*" mentioned above. This could be used to create the following (defective) partitioning:

- Order of file *F*
  - b_1 = Sorted in ascending order
  - b_2 = Sorted in descending order
  - b_3 = Arbitrary order

However, this is **not** a valid partitioning. Specifically, a file of length 0 or 1 belongs in all three blocks. That is, the blocks are not disjoint. The easiest strategy to address this problem is to make sure that each characteristic addresses only one property. The problem above is that the notions of being sorted into ascending order and being sorted into descending order are lumped into the same characteristic. Splitting into two characteristics, namely sorted ascending and sorted descending, solves the problem. The result is the following (valid) partitioning of two characteristics.

- File *F* sorted ascending
  - b_1 = True
  - b_2 = False

- File *F* sorted descending
  - b_1 = True
  - b_2 = False

With these blocks, files of length 0 or 1 are in the True block for both characteristics.

The completeness and disjointness properties are formalized for

pragmatic reasons, and not just to be mathematically fashionable. Two very different tasks are at the heart of testing with an input domain model: First, modeling the input domain, that is, choosing characteristics and partitions, and second, combining partitions into tests, that is, choosing a coverage criterion. It is extremely important to keep these tasks separate. An input domain model that prematurely encodes combination decisions is unnecessarily complex, and the resulting tests will almost certainly not reflect the combinations demanded by the coverage criterion. Fortunately, verifying the mathematical properties of completeness and disjointness help the test engineer separate these two tasks. That is, mixing these two tasks is the most common reason for partitions that are incomplete or have overlap. Conversely, characteristics with complete and pairwise-disjoint partitions generally are free of (inappropriate) combination decisions. In short, the mathematical checks guide the test engineer into making the right decisions. The rest of this chapter assumes that the partitions are both complete and disjoint.

6.1 INPUT DOMAIN MODELING

The first step in input domain modeling is to identify testable functions. Below is a signature for a method, `triang()`, that classifies triangles based on the lengths of the three sides. Source code for the class `TriangleType` (which contains the `triang()` method) is available on the book website, although we will not need it for this running example. The signature is enough.

```
public enum Triangle {Scalene, Isosceles, Equilateral, Invalid}
public static Triangle triang (int Side1, int Side2, int Side3)
// Side1, Side2, and Side3 represent the lengths of the sides of a triangle.
// Returns the appropriate enum value
```

Method `triang()` clearly has only one testable function with three parameters, which is common in unit testing. Finding the testable functions is more complex for Java class APIs. Each public method is typically a testable function that should be tested individually. However, the characteristics are often the same for several methods, so it helps to develop a common set of characteristics for the entire class and then

develop specific tests for each method. Finally, large systems are certainly amenable to the input space partition approach, and such systems often supply complex functionality. Modeling artifacts such as UML use cases can be used to identify testable functions. Each use case is associated with a specific intended functionality of the system, so it is very likely that the use case designers have useful characteristics in mind that are relevant to developing test cases. For example, a "withdrawal" use case for an ATM identifies "withdrawing cash" as a testable function. Further, it suggests useful characteristics such as "Is Card Valid?" and "Relation of Withdrawal Policy to Withdrawal Request."

The second step is to identify all of the parameters that can affect the behavior of a given testable function. This step isn't particularly creative, but it is important to carry it out completely. In the simple case of testing a stateless method, the parameters are simply the formal parameters to the method. If the method has state, which is common in many object-oriented classes, then the state must be included as a parameter. For example, the `add (E e)` method for a binary tree class such as Java's `TreeSet` behaves differently depending on whether or not `e` is already in the tree. Hence, the current state of the tree needs to be explicitly identified as a parameter to the `add()` method. In a slightly more complex example, a method `find (String str)` that finds the location of `str` in a file depends, obviously, on the file being searched. Hence, the test engineer explicitly identifies the file as a parameter to the `find()` method. Together, all of the parameters form the *input domain* of the function under test.

The third step, and the key creative engineering step, is modeling the input domain articulated in the prior step. An *input domain model* (*IDM*) represents the input space of the system under test in an abstract way. A test engineer describes the structure of the input domain in terms of input *characteristics*. The test engineer creates a *partition* for each characteristic. The partition is a set of *blocks*, each of which contains a set of *values*. From the perspective of that particular characteristic, all values in each block are considered equivalent.

A test input is a tuple of values, one for each parameter. By definition, the test input uses exactly one block from each characteristic. Thus, if we have even a modest number of characteristics, the number of possible combinations may be infeasible. In particular, adding another characteristic with *n* blocks increases the number of combinations by a

factor of *n*. Hence, controlling the total number of combinations is a key feature of any practical approach to input domain testing. In our view, this is the job of the coverage criteria, which we address in Section 6.2.

Different testers will come up with different models, depending on creativity and experience. These differences create a potential for variance in the quality of the resulting tests. The structured method to support input domain modeling presented in this chapter can decrease this variance and increase the overall quality of the IDM.

Once the IDM is built and values are identified, some combinations of the values may be invalid. The IDM must include information to help the tester identify and avoid or remove invalid sub-combinations. The model needs a way to represent these restrictions. Constraints are discussed further in Section 6.3.

The next section provides two different approaches to input domain modeling. The *interface-based* approach develops characteristics directly from input parameters to the program under test. The *functionality-based* approach develops characteristics from a functional or behavioral view of the program under test. The tester must choose which approach to use. Once the IDM is developed, several coverage criteria are available to decide which combinations of values to use to test the software. These are discussed in Section 6.2.

6.1.1 Interface-Based Input Domain Modeling

The interface-based approach considers each parameter separately. This approach is almost mechanical to follow, but the resulting tests are usually quite good.

An obvious strength of using the interface-based approach is that it is easy to identify characteristics. The fact that each characteristic limits itself to a single parameter also makes it easy to translate the abstract tests into executable test cases.

A weakness of this approach is that not all the information available to the test engineer will be reflected in the interface domain model. This means that the IDM may be incomplete and hence additional characteristics are needed.

Another weakness is that some parts of the functionality may depend on combinations of specific values of several interface parameters. In the

interface-based approach each parameter is analyzed in isolation with the effect that important sub-combinations may be missed.

Again, consider the `triang()` method. Its three integer parameters represent the lengths of three sides of a triangle. In an interface-based IDM, `Side1` will have a number of characteristics, as will `Side2` and `Side3`. Since the three variables are all of the same type, the interface-based characteristics for each will likely be identical. For example, since `Side1` is an integer, and zero is often a special value for integers, `Relation of Side1 to zero` is a reasonable interface-based characteristic.

6.1.2 Functionality-Based Input Domain Modeling

The idea of the functionality-based approach is to identify characteristics that correspond to the intended behavior, or functionality, of the system under test rather than using the actual interface. This allows the tester to incorporate some semantics or domain knowledge into the IDM.

Some members of the community believe that a functionality-based approach yields better test cases than the interface-based approach because the input domain models include more semantic information. Transferring more semantic information from the specification to the IDM makes it more likely to generate expected results for the test cases, an important goal.

Another important strength of the functionality-based approach is that the requirements are available before the software is implemented. This means that input domain modeling and test case generation can start early in development.

In the functionality-based approach, identifying characteristics and values may be far from trivial. If the system is large and complex, or the specifications are informal and incomplete, it can be very hard to design reasonable characteristics. The next subsection gives practical suggestions for designingcharacteristics.

The functionality-based approach also makes it harder to generate tests. The characteristics of the IDM often do not map to single parameters of the software interface. Translating the values into executable test cases is harder because constraints of a single IDM characteristic may affect multiple parameters in the interface.

Returning to the `triang()` method, a functionality-based approach will recognize that instead of simply three integers, the input to the method is a triangle. This leads to the characteristic of a triangle, which can be partitioned into different types of triangles (as discussed below).

6.1.3 Designing Characteristics

Designing characteristics in an interface-based approach is simple. There is a mechanical translation from the parameters to characteristics. Developing a functionality-based IDM is more challenging.

Preconditions are excellent sources for functionality-based characteristics. They may be explicit or encoded in the software as exceptional behaviors. Preconditions explicitly separate defined (or normal) behavior from undefined (or exceptional) behavior. For example, if a method `choose()` is supposed to select a value, it needs a precondition that a value must be available to select. A characteristic may be whether the value is available or not.

Postconditions are also good sources for characteristics. For the `triang()` method, the different kinds of triangles are based on the postcondition of the method.

The test engineer should also look for other relationships between variables. These may be explicit or implicit. For example, a curious test engineer given a method `m()` with two object parameters `x` and `y` might wonder what happens if `x` and `y` point to the same object (aliasing), or to logically equal objects. This is a form of stress testing.

Another possible idea is to check for missing factors, that is, factors that may impact the execution but do not have an associated IDM parameter.

Characteristics with few blocks are more likely to satisfy the disjointness and completeness properties. For this reason, it is often better to have many characteristics with few blocks than the inverse.

Generally, it is preferable for the test engineer to use specifications or other documentation instead of program code to develop characteristics. The idea is that the tester should apply input space partitioning by using *domain knowledge* about the problem, not the implementation. However, in practice, the code may be all that is available. Overall, the more semantic information the test engineer can incorporate into characteristics, the better the resulting test set is likely to be.

The two approaches generally result in different IDM characteristics. The following method illustrates this difference:

```
public boolean findElement (List list, Object element)
// Effects: if list or element is null throw NullPointerException
//    else returns true if element is in the list, false otherwise
```

If the interface-based approach is used, the IDM will have characteristics for `list` and characteristics for `element`. For example, here are two interface-based characteristics for `list`, including blocks and values, which are discussed in detail in the next section:

Characteristic	b_1	b_2
`list` is null	True	False
`list` is empty	True	False

The functionality-based approach results in more complex IDM characteristics. As mentioned earlier, the functionality-based approach requires more thinking on the part of the test engineer, but can result in better tests. Two possibilities for the example are listed below, again including blocks and values.

Characteristic	b_1	b_2	b_3
Number of occurrences of `element` in `list`	0	1	More than 1
`element` occurs first in `list`	True	False	
`element` occurs last in `list`	True	False	

6.1.4 Choosing Blocks and Values

After choosing characteristics, the test engineer partitions the domains of the characteristics into sets of values called *blocks*. A key issue in any partition approach is how partitions should be identified and how representative values should be selected from each block. This is another creative design step that allows the tester to tune the test process. More blocks will result in more tests, requiring more resources but possibly finding more faults. Fewer blocks will result in fewer tests, saving

resources but possibly reducing test effectiveness. Several general strategies for partitioning characteristics into blocks are given below. For any given characteristic one or two strategies are likely to be applicable.

- **Valid vs. invalid values**: Every partition must allow all values, whether valid or invalid. (This is simply a restatement of the completeness property.)
- **Sub-partition**: A range of valid values can often be partitioned into sub-partitions, such that each sub-partition exercises a somewhat different part of the functionality.
- **Boundaries**: Values at or close to boundaries often cause problems. This is a form of stress testing.
- **Normal use** (happy path) : If the operational profile focuses heavily on "normal use," the failure rate depends on values that are not boundary conditions.
- **Enumerated types**: A partition where blocks are a discrete, enumerated set often makes sense. The triangle example uses this approach.
- **Balance**: From a cost perspective, it may be cheap or even free to add more blocks to characteristics that have fewer blocks. In Section 6.2, we will see that the number of tests sometimes depends on the characteristic with the maximum number of blocks.
- **Missing blocks**: Check that the union of all blocks of a characteristic completely covers the input space of that characteristic.
- **Overlapping blocks**: Check that no value belongs to more than one block.

Special values can often be used. For a Java reference variable (that is, a pointer), `null` is typically a special case that needs to be treated differently from `non null` values. If the reference is to a container structure such as a `Set` or `List`, then whether the container is empty or not is often a useful characteristic.

Again consider the `triang()` method. It has three integer parameters that represent the lengths of three sides of a triangle. One common partitioning for an integer variable considers the relation of the variable's value to some special value in the testable function's domain, such as zero.

Table 6.1 shows a partitioning for the interface-based IDM for the `triang()` method. It has three characteristics, q_1, q_2, and q_3. The first

row in the table should be read as "Block $q_1.b_1$ is that Side 1 is greater than zero," "Block $q_1.b_2$ is that Side 1 is equal to zero," and "Block $q_1.b_3$ is that Side 1 is less than zero."

Table 6.1. First partitioning of `triang()`'s inputs (interface-based).

Partition	*b*1	*b*2	*b*3
q_1 = "Relation of Side 1 to 0"	*greater than 0*	*equal to 0*	*less than 0*
q_2 = "Relation of Side 2 to 0"	*greater than 0*	*equal to 0*	*less than 0*
q_3 = "Relation of Side 3 to 0"	*greater than 0*	*equal to 0*	*less than 0*

Consider the characteristic q_1 for Side 1. If one value is chosen from each block, the result is three tests. For example, we might choose Side 1 to have the value 7 in test 1, 0 in test 2, and -3 in test 3. Of course, we also need values for Side 2 and Side 3 of the triangle to complete the test case values. Notice that some of the blocks represent valid triangles and some represent invalid triangles. For example, no valid triangle can have a side of negative length.

It is easy to refine this categorization to get more fine-grained testing if the budget allows. For example, more blocks can be created by separating inputs with value 1. This decision leads to a partitioning with four blocks, as shown in Table 6.2.

Table 6.2. Second partitioning of `triang()`'s inputs (interface-based).

Partition	b_1	b_2	b_3	b_4
q_1 = "Length of Side 1"	*greater than 1*	*equal to 1*	*equal to 0*	*less than 0*
q_2 = "Length of Side 2"	*greater than 1*	*equal to 1*	*equal to 0*	*less than 0*
q_3 = "Length of Side 3"	*greater than 1*	*equal to 1*	*equal to 0*	*less than 0*

Notice that if the value for Side 1 were floating point rather than integer, the second categorization would **not** yield valid partitions. None of the blocks would include values between 0 and 1 (non-inclusive), so the blocks would not cover the domain (not be complete). However, the domain *D* contains integers so the partitions are valid.

While partitioning, it is often useful for the tester to identify candidate values for each block to be used in testing. The reason to identify values now is that choosing specific values can help the test engineer think more

concretely about the predicates that describe each block. While these values may not prove sufficient when refining test requirements to test cases, they do form a good starting point. Table 6.3 shows values that can satisfy the second partitioning.

Table 6.3. Possible values for blocks in the second partitioning in Table 6.2.

Parameter	b_1	b_2	b_3	b_4
Side 1	2	1	0	-1
Side 2	2	1	0	-1
Side 3	2	1	0	-1

The above partitioning is interface-based and only uses syntactic information about the program (it has three integer inputs). A functionality-based approach can use the semantic information of the traditional geometric classification of triangles, as shown in Table 6.4.

Table 6.4. Geometric partitioning of `triang()`'s inputs (functionality-based).

Partition	b_1	b_2	b_3	b_4
q_1 = "Geometric Classification"	*scalene*	*isosceles*	*equilateral*	*invalid*

Of course, the tester has to know what makes a triangle scalene, equilateral, isosceles, and invalid to choose possible values (this may be middle school geometry, but many of us have probably forgotten). An *equilateral* triangle is one in which all sides are the same length. An *isosceles* triangle is one in which at least two sides are the same length. A *scalene* triangle is any other valid triangle. This brings up a subtle problem —Table 6.4 does **not** form a valid partitioning. An equilateral triangle is also isosceles, thus we must first correct the partitions, as shown in Table 6.5.

Table 6.5. Correct geometric partitioning of `triang()`'s inputs (functionality-based).

Partition	b_1	b_2	b_3	b_4
q_1 = "Geometric Classification"	*scalene*	*isosceles, not equilateral*	*equilateral*	*invalid*

Now values for Table 6.5 can be chosen as shown in Table 6.6. The triplets represent the three sides of the triangle.

Table 6.6. Possible values for blocks in geometric partitioning in Table 6.5.

Param	b_1	b_2	b_3	b_4
Triangle	(4, 5, 6)	(3, 3, 4)	(3, 3, 3)	(3, 4, 8)

A different approach to the equilateral/isosceles problem above is to break the characteristic `GeometricPartitioning` into four separate characteristics, namely `Scalene`, `Isosceles`, `Equilateral`, and `Valid`. The partition for each of these characteristics is boolean, and the fact that choosing `Equilateral = true` also means choosing `Isosceles = true` is then simply a constraint. We recommend such an approach for this example: It invariably satisfies the disjointness and completeness properties.

6.1.5 Checking the Input Domain Model

It is important to check the input domain model. In terms of characteristics, the test engineer should ask whether any information about how the function behaves is not incorporated in some characteristic. This is necessarily an informal process.

The tester should also explicitly check each characteristic for the completeness and disjointness properties. The purpose of this check is to make sure that, for each characteristic, not only do the blocks cover the complete input space, but selecting a particular block implies excluding all other blocks in that characteristic.

If multiple IDMs are used, completeness should be relative to the portion of the input domain that is modeled in each IDM. When the tester is satisfied with the characteristics and their blocks, it is time to choose which combinations of values to test with and identify constraints among the blocks.

EXERCISES

Section 6.1.

1. Return to the example at the beginning of the chapter of the two characteristics "File F sorted ascending" and "File F sorted descending." Each characteristic has two blocks. Give test case values for all four combinations of these two characteristics.
2. A tester defined three characteristics based on the input parameter *car*: **Where Made**, **Energy Source**, and **Size**. The following partitionings for these characteristics have at least two mistakes. Correct them.

Where Made		
North America	Europe	Asia
Energy Source		
gas	electric	hybrid
Size		
2-door	4-door	hatch back

3. Answer the following questions for the method `search()` below:

```
public static int search (List list, Object element)
// Effects: if list or element is null throw NullPointerException
//   else if element is in the list, return an index
//   of element in the list; else return -1
//   for example, search ([3,3,1], 3) = either 0 or 1
//       search ([1,7,5], 2) = -1
```

Base your answer on the following characteristic partitioning:

```
Characteristic: Location of element in list
    Block 1: element is first entry in list
    Block 2: element is last entry in list
    Block 3: element is in some position other than first or last
```

(a) "Location of element in list" fails the disjointness property. Give an example that illustrates this.

(b) "Location of element in list" fails the completeness property. Give an example that illustrates this.

(c) Supply one or more new partitions that capture the intent of "Location of element in list" but do not suffer from completeness or disjointness problems.

4. Derive input space partitioning test inputs for the `GenericStack` class assuming the following method signatures:
   - `public GenericStack ();`
   - `public void push (Object X);`
   - `public Object pop ();`
   - `public boolean isEmpty ();`

   Assume the usual semantics for the `GenericStack`. Try to keep your partitioning simple and choose a small number of partitions and blocks.
   (a) List all of the input variables, including the state variables.
   (b) Define characteristics of the input variables. Make sure you cover all input variables.
   (c) Define characteristics of inputs.
   (d) Partition the characteristics into blocks.
   (e) Define values for each block.
5. Consider the problem of searching for a pattern string in a subject string. One possible implementation with a specification is on the book website; `PatternIndex.java`. This version has an incomplete specification–and a good interface-based input domain model should single out the problematic input! Assignment: find the problematic input, complete the specification, and revise the implementation to match the revised specification.

6.2 COMBINATION STRATEGIES CRITERIA

The discussion in Section 6.1 skips an important question: "How should we consider multiple partitions at the same time?" This is the same as asking "What combination of blocks should we choose values from?" For example, we might wish to require a test case that satisfies block 1 from q_2 and block 3 from q_3. The most obvious choice is to choose all combinations. However, using all combinations will be impractical when more than two or three partitions are defined.

CRITERION **6.1 All Combinations Coverage (ACoC):** *All combinations of blocks from all characteristics must be used.*

For example, if we have three partitions with blocks [A, B], [1, 2, 3], and [x, y], then ACoC will need the following twelve tests:

(A, 1, x)	(B, 1, x)
(A, 1, y)	(B, 1, y)
(A, 2, x)	(B, 2, x)
(A, 2, y)	(B, 2, y)
(A, 3, x)	(B, 3, x)
(A, 3, y)	(B, 3, y)

A test set that satisfies ACoC will have a unique test for each combination of blocks for each partition. The number of tests is the product of the number of blocks for each partition: $\prod_{i=1}^{Q}(B_i)$.

If we use a four block partition similar to q_2 for each of the three sides of the triangle, ACoC requires 4 * 4 * 4 = 64 tests.

This is almost certainly more testing than is necessary, and will usually be economically impractical as well. Thus, we must use some sort of coverage criterion to choose which combinations of blocks to pick values from.

The first, fundamental, assumption is that different choices of values from the same block are equivalent from a testing perspective. That is, we need to use only one value from each block. Several *combination strategies* exist, which result in a collection of useful criteria. These combination strategies are illustrated with the `triang()` example, using the second categorization given in Table 6.2 and the values fromTable 6.3.

The first combination strategy criterion is fairly straightforward and simply requires that we try each choice at least once.

CRITERION **6.2 Each Choice Coverage (ECC):** *One value from each block for each characteristic must be used in at least one test case.*

Given the above example of three partitions with blocks [A, B], [1, 2, 3], and [x, y], ECC can be satisfied in many ways, including the three tests (A, 1, x), (B, 2, y), and (A, 3, x).

Assume the program under test has Q characteristics q_1, q_2, . .., q_Q, and each characteristic q_i has B_i blocks. Then a test set that satisfies ECC will have at least $Max_{i=1}{}^{Q}B_i$ values. The maximum number of blocks for the partitions for `triang()` is four, thus ECC requires at least four tests.

This criterion can be satisfied on `triang()` by choosing the tests {(2, 2, 2), (1, 1, 1), (0, 0, 0), (-1, -1, -1)} fromTable 6.3. It does not take much thought to conclude that these are not very effective tests for this program. ECC leaves a lot of flexibility to the tester in terms of how to combine the test values, so it can be called a relatively "weak" criterion.

The weakness of ECC can be expressed as not requiring values to be combined with other values. A natural next step is to require explicit combinations of values, called *pair-wise*.

CRITERION **6.3 Pair-Wise Coverage (PWC):** *A value from each block for each characteristic must be combined with a value from every block for each other characteristic.*

Given the above example of three partitions with blocks [A, B], [1, 2, 3], and [x, y], then PWC will need tests to cover the following 16 combinations:

(A, 1)	(B, 1)	(1, x)
(A, 2)	(B, 2)	(1, y)
(A, 3)	(B, 3)	(2, x)
(A, x)	(B, x)	(2, y)
(A, y)	(B, y)	(3, x)
		(3, y)

PWC allows the same test case to cover more than one unique pair of values. So the above combinations can be combined in several ways, including:

(A, 1, x)	(B, 1, y)
(A, 2, x)	(B, 2, y)
(A, 3, x)	(B, 3, y)
(A, -, y)	(B, -, x)

The tests with '-' mean that any block can be used.

A test set that satisfies PWC will pair each value with each other value, or have at least($Max_{i=1}^{Q} B_i$) * ($Max_{i=1, j=1}^{Q} B_j$) values. Each characteristic in `triang()` (Table 6.3) has four blocks; so at least 16 tests are required.

Several algorithms to satisfy PWC have been published and appropriate references are provided in the bibliography section of the chapter.

A natural extension to Pair-Wise Coverage is to require groups of *t* values instead of pairs.

> CRITERION **6.4 T-Wise Coverage (TWC):** *A value from each block for each group of t characteristics must be combined.*

If the value for *t* is chosen to be the number of partitions, *Q*, then T-Wise Coverage is equivalent to all combinations. If we assume that all blocks are the same size, a test set that satisfies TWC will have at least($Max_{i=1}^{q} B_i$)^t values. T-Wise Coverage is expensive in terms of the number of test cases, and experience suggests going beyond pair-wise (that is, $t = 2$) does not help much.

Both Pair-Wise Coverage and T-Wise Coverage combine values "blindly," without regard for which values are being combined. The next criterion strengthens ECC in a different way by bringing in a small but crucial piece of domain knowledge of the program; asking what is the most "important" block for each partition. This block is called the *base choice*.

> CRITERION **6.5 Base Choice Coverage (BCC):** *A base choice block is chosen for each characteristic, and a base test is formed by using the base choice for each characteristic. Subsequent tests are chosen by holding all but one base choice constant and using each non-base choice in each other characteristic.*

Given the above example of three partitions with blocks [A, B], [1, 2, 3], and [x, y], suppose base choice blocks are 'A', '1' and 'x'. Then the base choice test is (A, 1, x), and the following additional tests would be needed:

(B, 1, x)
(A, 2, x)

(A, 3, x)
(A, 1, y)

A test set that satisfies BCC will have one base test, plus one test for each remaining (non-base) block for each partition. This is a total of $1 + \sum_{i=1}^{Q}(B_i - 1)$. Each parameter for `triang()` has four blocks, thus BCC requires 1 + 3 + 3 + 3 tests.

The base choice can be the simplest, the smallest, the first in some ordering, or the most likely from an end-user point of view. Combining more than one invalid value is usually not useful because the software often recognizes one value and negative effects of the others are masked. Which blocks are chosen for the base choices becomes a crucial step in test design that can greatly impact the resulting test. For example, if the base choice contains valid inputs and most or all of the non-base choices are invalid, then BCC is an easy way to achieve stress testing. It is important that the tester document the strategy that was used so that further (regression) testers can re-evaluate those decisions.

Following the strategy of choosing the most likely block for `triang()`, we chose "greater than 1" from Table 6.2 as the base choice block. Using the values from Table 6.3 gives the base test as (2, 2, 2). The remaining tests are created by varying each one of these in turn: {(2, 2, 1), (2, 2, 0), (2, 2, -1), (2, 1, 2), (2, 0, 2), (2, -1, 2), (1, 2, 2), (0, 2, 2), (-1, 2, 2)}.

Sometimes the tester may have trouble choosing a single base choice and may decide that multiple base choices are needed. This is formulated as follows:

CRITERION **6.6 Multiple Base Choice Coverage (MBCC):** *At least one, and possibly more, base choice blocks are chosen for each characteristic, and base tests are formed by using each base choice for each characteristic at least once. Subsequent tests are chosen by holding all but one base choice constant for each base test and using each non-base choice in each other characteristic.*

Assuming m_i base choices for each characteristic and a total of M base tests, MBCC requires $M + \sum_{i=1}^{Q}(M * (B_i - m_i))$ tests.

For example, we may choose to include two base choices for side 1 in

`triang ()`, "greater than 1" and "equal to 1." This would result in the two base tests (2, 2, 2) and (1, 2, 2). The formula above is thus evaluated with $M = 2$, $m_1 = 2$, and $m_i = 1 \ \forall \ i, 1 < i \leq 3$. That is, 2 + (2*(4-2)) + (2*(4-1)) + (2*(4-1)) = 18. The remaining tests are created by varying each one of these in turn. The MBCC criterion sometimes results in duplicate tests. For example, (0, 2, 2) and (-1, 2, 2) both appear twice for `triang ()`. Duplicate test cases should, of course, be eliminated (which also makes the formula for the number of tests an upper bound).

Figure 6.2 shows the subsumption relationships among the input space partitioning combination strategy criteria.

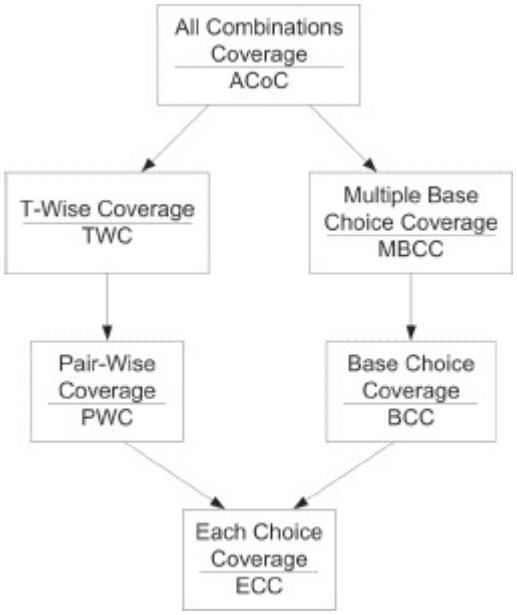

Figure 6.2. Subsumption relations among input space partitioning criteria.

EXERCISES

Section 6.2.

1. Write down all 64 tests to satisfy the All Combinations (ACoC) criterion for the second categorization of `triang()`'s inputs in Table 6.2. Use the values in Table 6.3.
2. Write down all 16 tests to satisfy the Pair-Wise (PWC) criterion for the second categorization of `triang()`'s inputs in Table 6.2. Use the values in Table 6.3.
3. Write down all 16 tests to satisfy Multiple Base Choice coverage (MBCC) for the second categorization of `triang()`'s inputs in

Table 6.2. Use the values in Table 6.3.

4. Answer the following questions for the method `intersection()` below:

```
public Set intersection (Set s1, Set s2)
  // Effects:  If s1 or s2 is null throw NullPointerException
  //    else return a (non null) Set equal to the intersection
  //    of Sets s1 and s2

  Characteristic:  Validity of s1
    - s1 = null
    - s1 = {}
    - s1 has at least one element

  Characteristic:  Relation between s1 and s2
    - s1 and s2 represent the same set
    - s1 is a subset of s2
    - s2 is a subset of s1
    - s1 and s2 do not have any elements in common
```

(a) Does the partition "Validity of s1" satisfy the completeness property? If not, give a value for *s*1 that does not fit in any block.

(b) Does the partition "Validity of s1" satisfy the disjointness property? If not, give a value for *s*1 that fits in more than one block.

(c) Does the partition "Relation between s1 and s2" satisfy the completeness property? If not, give a pair of values for *s*1 and *s*2 that does not fit in any block.

(d) Does the partition "Relation between s1 and s2" satisfy the disjointness property? If not, give a pair of values for *s*1 and *s*2 that fits in more than one block.

(e) If the "Base Choice" criterion were applied to the two partitions (exactly as written), how many test requirements would result?

(f) Revise the characteristics to eliminate any problems you found.

5. Use the following characteristics and blocks for the questions below.

Characteristics	Block 1	Block 2	Block 3	Block 4
Value 1	< 0	0	> 0	
Value 2	< 0	0	> 0	
Operation	+	−	×	÷

(a) Give tests to satisfy the *Each Choice* criterion.
(b) Give tests to satisfy the *Base Choice* criterion. Assume base choices are *Value 1* = > 0, *Value 2* = > 0, and *Operation* = +.
(c) How many tests are needed to satisfy the *All Combinations* criterion? (Do not list all the tests.)
(d) Give tests to satisfy the *Pair-Wise Coverage* criterion.

6. Derive input space partitioning test inputs for the **BoundedQueue** class with the following method signatures:

```
public BoundedQueue (int capacity); // The maximum number of elements
public void enQueue (Object X);
public Object deQueue ();
public boolean isEmpty ();
public boolean isFull ();
```

Assume the usual semantics for a queue with a fixed, maximal capacity. Try to keep your partitioning simple—choose a small number of partitions and blocks.
(a) List all of the input variables, including the state variables.
(b) Define characteristics of the input variables. Make sure you cover all input variables.
(c) Partition the characteristics into blocks. Designate one block in each partition as the "Base" block.
(d) Define values for each block.
(e) Define a test set that satisfies Base Choice Coverage (BCC). Write your tests with the values from the previous step. Be sure to include the test oracles.

7. Design an input domain model for the logic coverage web application on the book's website. That is, model the logic coverage web application using the input domain modeling technique.
(a) List all of the input variables, including the state variables.
(b) Define characteristics of the input variables. Make sure you cover all input variables.
(c) Partition the characteristics into blocks.
(d) Designate one block in each partition as the "Base" block.
(e) Define values for each block.
(f) Define a test set that satisfies Base Choice Coverage (BCC). Write your tests with the values from the previous step. Be sure

to include the test oracles.

(g) Automate your tests using the web test automation framework *HttpUnit*. Demonstrate success by submitting the HttpUnit tests and a screen dump or output showing the result of execution.
(Note to instructors: HttpUnit is based on JUnit and is quite similar. The tests must include a URL and the framework issues the appropriate HTTP request. We usually use this question as a non-required bonus, allowing students to choose whether to learn HttpUnit on their own.)

6.3 HANDLING CONSTRAINTS AMONG CHARACTERISTICS

A subtle point about input space partitioning is that some combinations of blocks are infeasible. This must be documented in the IDM. Table 6.7 shows an example based on the previously described `boolean findElement(list, element)` method. An IDM with two characteristics, *A*, which has four blocks, and *B*, which has three blocks, has been designed. Two of the block combinations do not make sense and are thus invalid. In this example, these are represented as a list of invalid pairs of characteristic blocks. Other representations can also be used, for example, a set of inequalities.

Table 6.7. Examples of invalid block combinations.

Characteristics	Blocks			
	1	2	3	4
A: length and contents	one element	more than one, unsorted	more than one, sorted	more than one, all identical
B: match	element not found	element found once	element found more than once	–

Invalid combinations: (A1, B3), (A4, B2)

Generally, *constraints* are relations between blocks from different characteristics. IDMs have two broad kinds of constraints. The first says that a block from one characteristic cannot be combined with a block from another characteristic. The “less than zero” and “scalene” problem for

`triang()` is an example of this kind of constraint. The second is the inverse; a block from one characteristic **must be** combined with a specific block from another characteristic. Although this sounds simple enough, identifying and satisfying the constraints when choosing values can be difficult.

How constraints are handled when selecting values depends on the coverage criterion chosen, and the decision is usually made when values are chosen. For the ACoC, PWC, and TWC criteria, the only reasonable option is to drop the infeasible pairs from consideration. For example, if PWC requires a particular pair that is not feasible, no amount of tinkering on the test engineer's part can make that test requirement feasible. However, the situation is quite different for BCC and MBCC. If a particular variation (for example, "less than zero" for "Relation of Side 1 to zero") conflicts with the base case (for example, "scalene" for "Geometric Classification"), then we can change the choice for the base case to make the test requirement feasible. In this case, "Geometric Classification" can be changed to "invalid."

We have one more point about constraints among partitions. If the IDM has too many constraints, it probably has a structural problem and should be redesigned.

6.4 EXTENDED EXAMPLE: DERIVING AN IDM FROM JAVADOC

This subsection works a complete example of constructing an IDM and designing tests for a widely used Java library interface. A common goal of JavaDoc is to have enough information for a tester to create tests. It turns out that input domain modeling is an excellent way to analyze a JavaDoc API to design test cases. This example takes a standard Java interface, `java.util.Iterator` [2], and uses it to design an IDM. We then apply a combination strategy criterion to the IDM, creating test requirements, which are then implemented in JUnit.

APIs are usually clear about what is testable and what the parameters are. If your API is not clear, your problems are probably deeper than finding good test sets. This would be a good time to talk with the software designer about the API and its documentation.

6.4.1 Tasks in Designing IDM-Based Tests

We create JUnit tests from JavaDoc APIs in three main tasks, each of which is decomposed into several steps.

The first task is to identify characteristics from the API, which we document in two tables, table A to identify the characteristics, and table B to associate the methods with the characteristics to identify test requirements. The second task designs test cases from the test requirements. If base choices are defined (when BCC or MBCC is used), those are documented in table B. The test requirements are expressed in a third table, C. The third task converts the test cases into automatable test scripts. These tasks are detailed as follows:

Task 1: Determine Characteristics

1. Identify the following and document them in Table A:
   - functional units
   - parameters
   - return types and possible return values
   - exceptional behavior
2. Using the method attributes identified in step 1, develop characteristics that would cover return types and exceptional behavior. Document the characteristics in Table A. Characteristics identified in some methods might cause you to revisit methods analyzed earlier, so this analysis is done iteratively. Other methods might indicate traits that have already been covered by a previously identified characteristic. This is documented in Table A by putting the previous characteristic in a “Covered by” column. This can cause you to come back and revisit methods analyzed earlier, so this analysis is also iterative.
3. As the above steps are being executed, associate each method with relevant characteristics in Table B.
4. Design a partitioning for each characteristic. This is documented in Table B.

Task 2: Define test requirements

1. Choose a coverage criterion from section 6.2.
2. Choose base cases if using BCC or MBCC, and document these in

Table B.
3. Design complete test requirements. This is documented in Table C.
4. Identify any infeasible test requirements (constraints), and document those in Table C.
5. If using BCC or MBCC, revise any infeasible tests to create feasible tests, and document them in Table C.

Task 3: Refine test requirements into automated tests

1. Using the final test requirements, write a JUnit test for each feasible test requirement. Each test requirement must map to one test. Although it would be possible to satisfy several test requirements with one test, we choose not to do that in this example to keep the tests and the mappings simple to follow.

6.4.2 Designing IDM-Based Tests for `Iterator`

Now we illustrate these steps and fill out the three tables with the `Iterator` JavaDoc. An Iterator can be defined on any collection, such as arrays, lists, stacks, queues, etc. An `Iterator` is sometimes said to contain "an iteration," and sometimes said to contain "a collection." `Iterators` are defined for generic types, so the type of the underlying collection is simply called `E`.

Task 1: Determine the characteristics of `Iterator`

1. The `Iterator` interface has three methods, none of which has an explicit parameter:
   - `hasNext()` – Returns true if the iteration has more elements.
   - `E next()` – Returns the next element in the iteration. The method has one possible exception, `NoSuchElementException`.
   - `void remove()` – Removes the most recent element returned by the iterator from the underlying collection. The method can throw two exceptions, `UnsupportedOperationException` and `IllegalStateException`.

The method names and signature elements (parameters, return types, and return values) are shown in Table 6.8. The "Parameters" column shows that the behavior of all three methods is determined by the "iterator state," which is surprisingly complex. While we are not concerned with how this state is implemented, we are definitely concerned with the information it contains. First, the state contains the values yet to be returned by the iterator via subsequent `next()` calls. Note that if there are no such values, `hasNext()` will return false. Second, the state contains information about whether `remove()` can be successfully called, and, if so, which object is to be removed, and from what underlying collection. Hence, the underlying collection is also part of the iterator state, a subtle point to which we return later in the example.

Table 6.8. Table A for `Iterator` example: Input parameters and characteristics.

Method Name	Parameters	Return Types	Possible Values	Exceptions	Characteristic ID	Characteristic	Covered by
hasNext()	Iterator state	boolean	true, false		C1	Iterator has more values	
next()	Iterator state	E(element) – generic type	E, null		C2	Iterator returns a non-null object reference	
				NoSuchElement-Exception			C1
remove()	Iterator state			UnsupportedOperationException	C3	remove() is supported	
				IllegalState-Exception	C4	remove() constraint is satisfied	

2. Develop the characteristics to test `Iterator`.
   - `hasNext()` returns a boolean value. This suggests that we would like a characteristic (C1) that forces both possible values.
   - `next()` returns an object of generic type, the type stored in the underlying collection. The most basic syntactic question is whether this value might be null, which is shown in Table 6.8 as category C2. `NoSuchElementException` will be thrown when the iterator has no more elements, which is already covered by characteristic C1 for `hasNext()`.
   - `remove()` has no explicit return values, which poses an observability problem, which we can solve through observer

methods on the underlying collection. `remove()` can also be tested through possible exceptions. The `Iterator` specification does not require `remove()` to be implemented, so `UnsupportedOperationException` is returned if the iterator does not support `remove()`. Thus characteristic C3 is defined to check whether `remove()` is supported. The other exception is `IllegalStateException`, so we add characteristic C4 to ensure the constraint of `remove()`, which is that the method "can be called only once per call to `next()`."

3. The methods and their characteristics are associated in the first four columns of Table 6.9 (Table B).
4. To partition each characteristic into blocks we use boolean partitions, as shown in the fifth column of Table 6.9.

Table 6.9. Table B for `Iterator` example: Partitions and base case.

	hasNext()	next()	remove()	Partition (all boolean)	Base Case
C1	X	X	X	{true, false}	true
C2		X	X	{true, false}	true
C3			X	{true, false}	true
C4			X	{true, false}	true

Task 2: Define the test requirements for `Iterator`

Table B is based on the analysis from the final step in Task 1 and all of Task 2.

1. We choose base choice coverage for this example.
2. We make the base case a "happy path" test, where everything should work normally with no exceptions. This is documented in Table 6.9 by choosing the true block from each partition.
3. This information is converted into test requirements in Table 6.10. For each method, its characteristics are listed, followed by the test requirements that are needed. The only characteristic for `hasNext()` is C1, so it has only two test requirements. The first is the base case where the iterator has more values (shown in bold face), and the second is the non-base case. `next()` has two characteristics, so each test must choose a value from each of C1 and C2. Again, the

first test requirement is the base case where both C1 and C2 are true, and then each is varied in turn to create three test requirements. `remove()` has four characteristics, so five test requirements are needed.

4. Next we identify infeasible test requirements. `hasNext()` has none, but `next()` has one. If C1 is false, the iterator is out of elements, so the iterator cannot return a non-null object reference, meaning C2 cannot be true. The same problem occurs in the second test for `remove()`.
5. Next we revise the infeasible test requirements to make them feasible by applying the fix suggested in Section 6.3 of modifying a non-base choice. For `next()`, we change the test requirement from FT to FF, and for `remove()` we change FTTT to FFTT.

Table 6.10. Table C for `Iterator` example: Refined test requirements.

Method	Characteristics	Test Requirements	Infeasible Test Requirements	Revised Test Requirements	# of Test Requirements
hasNext()	C1	{**T**, F}	All feasible	n/a	2
next()	C1 C2	{**TT**, FT, TF}	FT	FT → FF	3
remove()	C1 C2 C3 C4	{**TTTT**, FTTT, TFTT, TTFT, TTTF}	FTTT	FTTT → FFTT	5

We could certainly do more testing. For example, does `next()` behave properly after something has been removed? However, the tests designed and presented in this section cover all of the explicit items in the JavaDoc.

Task 3: Refine Tests into Automated Tests

The result at this point is two test requirements for `hasNext()`, three test requirements for `next()`, and five test requirements for `remove()`. Each test needs to be refined into a concrete test case, including a verification of the output (an oracle).

`Iterator` is simply an interface, and Java interfaces are not directly executable. Hence, to develop concrete tests, we need an implementation of `Iterator`. In a realistic testing situation, it would be obvious which implementation we were testing–namely the implementation we were developing! For this exercise, we choose a standard Java implementation

of `Iterator` simply to show how the tests can be implemented.

The specific implementation chosen has a major impact on the feasibility of implementing the tests. For example, `null` values are possible in the `ArrayList` class but not in the `TreeSet` class. Thus, our tests (mostly) target the `ArrayList` implementation of `Iterator`.

The complete set of JUnit tests takes about 150 lines of Java, so we do not include them in the text. The complete JUnit, `IteratorTest.java`, can be found on the book website. We provide a few examples here to illustrate the process and controllability and observability issues. The tests also reveal something interesting about how Java `Collection` classes treat inconsistencies.

Test `testHasNext_BaseCase()` is our first test. Characteristic C1 is true. The test fixture for all of the JUnit tests has two variables: a `List` of strings and an `Iterator` for strings. The `@Before` method `setUp()` sets up the test fixture so that the `list` variable holds an `ArrayList` with two strings, and the `itr` variable is initialized to iterate over the list. The `setUp()` method defines the prefix values common to most of the tests. The test itself simply calls `itr.hasNext()` (the test case value). The test checks the result by asserting the return value from `hasNext()` is true (verification value).

```
private List<String> list;          // test fixture
private Iterator<String> itr;       // test fixture

@Before public void setUp()         // set up test fixture
{
  list = new ArrayList<String>();
  list.add("cat");

  list.add("dog");
  itr = list.iterator();
}

// Test 1 of hasNext(): testHasNext_BaseCase():  C1-T
@Test public void testHasNext_BaseCase()
{
  assertTrue(itr.hasNext());
}
```

Our second example is slightly more complicated. This tests for when C3 is false, that is, the iterator does not support the `remove()` method.

To make this happen, this test turns `list` into an `unmodifiableList`, which, as its name implies, is a list that cannot be changed. The `remove()` method must not be implemented when iterating over such a structure, and so when `remove()` is called, an `UnsupportedOperationException` should be raised. Finding a suitable unmodifiable list is an example of solving a subtle controllability problem. Note that this JUnit test does not have an assertion statement. Instead, the " `expected`" attribute in the `@Test` annotation is used to indicate it should return an exception. The test passes if the exception is returned, and fails otherwise. See Chapter 3 for a fuller discussion of this point.

```
// Test 4 of remove(): testRemove_C3(): C1-T, C2-T, C3-F, C4-T
@Test(expected=UnsupportedOperationException.class)
public void testRemove_C3()
{
  list = Collections.unmodifiableList(list);
  itr = list.iterator();
  itr.remove();
}
```

The third example is another test for `remove()`. The `remove()` method has a complex constraint in terms of how it must be interleaved with calls to `next()`. If `remove()` is called in a state that does not satisfy this constraint, then the `remove()` method must return an `IllegalStateException`. Put into the language of design-by-contract theory, what was once a precondition (i.e., undefined behavior) on how calling code must interleave `remove()` with `next()` has been converted into a postcondition (i.e., defined behavior) through the exception handling mechanism. Although this constraint is complex, we only test it with one test here; additional tests are left for the exercises. The test below calls `remove()` without calling `next()` at all.

```
//  Test 5 of remove(): testRemove_C4(): C1-T, C2-T, C3-T, C4-F
@Test(expected=IllegalStateException.class)
public void testRemove_C4()
{
  itr.remove();
}
```

Analysis and iteration: Another possible exception

Finally, the documentation for `remove()` contains a precondition on the use of the iterator. Specifically, the specification says "*The behavior of an iterator is unspecified if the underlying collection is modified while the iteration is in progress in any way other than by calling this method.*" That is, `remove()` is the only allowable way to change the underlying collection while the iterator is in use, and, for example, if an element is added to the collection while the iterator is in use, bad things might happen. The phrasing "the behavior. .. is unspecified" implies a genuine precondition: a correct iterator can implement *any* behavior, including silently ignoring the call, corrupting the data structure, returning arbitrary values, or even failing to terminate.

As testers, we could certainly stop with the above tests and claim to have made a "reasonable effort" to test the iterator. We could also assert that since the "behavior is unspecified," any behavior is okay so there is no need to test. However, if we have a true tester's mindset, we should worry about preconditions –they are a prime weapon in security hacker's toolkits[3]. In short, there is a reasonable argument to be made that we should be concerned about what happens if the collection is modified and the iterator is subsequently used.

Many standard implementations of Java iterators transform this precondition into defined behavior with `ConcurrentModificationException`. In particular, Java `Collection` classes, of which `List` is a member, use this exception. Thus we can add another characteristic, C5, called `ConcurrentModificationException`. This characteristic is obviously associated with `remove()`, but if we consider carefully, we might also recognize that `hasNext()` and `next()` can also be affected by a change to the underlying collection. Thus we chose to associate C5 with all three methods, as reflected in a revised version of Table 6.8 (Table A), which is shown in Table 6.11.

Table 6.11. Table A for `Iterator` example: Input parameters and characteristics (revised).

Method Name	Param-eters	Return Types	Possible Values	Exceptions	Charact-eristic ID	Characteristic	Covered by
hasNext()	iterator state	boolean	true, false		C1	iterator has more values	
				ConcurrentModif-icationException			C5
next()	iterator state	E(element) – generic type	E, null		C2	iterator returns a non-null object reference	
				NoSuchElement-Exception			C1
				ConcurrentModif-icationException			C5
remove()	iterator state			UnsupportedOper-ationException	C3	remove() is supported	
				IllegalState-Exception	C4	remove() precondition is satisfied	
				ConcurrentModif-icationException	C5	collection unmodified while iterator in use	

Considering `ConcurrentModificationException` only adds one more row to Table 6.9, so we do not show a revised Table B. However, a new characteristic that applies to every testable method affects every test requirement, as well as adding new ones, so we revise Table 6.10 as Table 6.12. The base case for `ConcurrentModificationException` is true; the iterator remains in a consistent state while the iterator is in use. Note that, as mentioned earlier, this analysis shows that the underlying collection is indeed part of the "iterator state" identified in Table A.

Table 6.12. Table C for `Iterator` example: Refined test requirements (revised).

Method	Characteristics	Test Requirements	Infeasible Test Requirements	Revised Test Requirements	# of Test Requirements
hasNext()	C1 C5	{**TT**, FT, TF}	All feasible	n/a	3
next()	C1 C2 C5	{**TTT**, FTT, TFT, TTF}	FTT TTF	FTT → FFT TTF → TFF	4
remove()	C1 C2 C3 C4 C5	{**TTTTT**, FTTTT, TFTTT, TTFTT, TTTFT, TTTTF}	FTTTT	FTTTT → FFTTT	6

Now we have three test requirements for `hasNext()`, the base case, plus one where C1 is false, and another where C5 is false. Likewise, we have an additional test for `next()` and for `remove()`.

Our final example JUnit test for the `Iterator` interface is a test for

remove() where the iterator is in an inconsistent state, and hence ConcurrentModificationException is expected. In testRemove_C5(), we solve the controllability problem by adding an element to the list. Note that this happens after the setUp(), which initializes the iterator. This puts the iterator in an inconsistent state, and the subsequent call to itr.remove() should throw ConcurrentModificationException.

```
// Test 6 for next(): testRemove_C5(): C1-T, C2-T, C3-T, C4-T, C5-F
@Test(expected=ConcurrentModificationException.class)
public final void testRemove_C5()
{
  itr.next();
  list.add ("elephant");
  itr.remove();
}
```

This JUnit test passes, as does a similar test where next() replaces remove(). (See test testNext_C5 in the online tests.) But a test where hasNext() replaces remove() does **not** pass. (See test testHasNext_C5 in the online tests.)

We use the Iterator example not only because it illustrates many of the interesting aspects of using input domain modeling to design tests from JavaDoc, but also because it illustrates the power of these tests. In our judgment, testHasNext_C5 demonstrates a flaw in Java Iterator implementations. The internal state of a data structure is either consistent or inconsistent. It makes no sense for a call to hasNext() to come back "true," and then for an immediate call to next() to throw ConcurrentModificationException. See the exercises for a variation where remove() fails to throw an expected ConcurrentModificationException.

EXERCISES

Section 6.4.

1. The restriction on interleaving next() and remove() calls is quite complex. The JUnit tests in IteratorTest.java only devote one

test for this situation, which may not be enough. Refine the input domain model with one or more additional characteristics to probe this behavior, and implement these tests in JUnit.

2. (**Challenging!**) It is possible to modify an `ArrayList` without using the `remove()` method and yet have a subsequent call to `remove()` **fail** to throw `ConcurrentModificationException`. Develop a (failing) JUnit test that exhibits this behavior.

6.5 BIBLIOGRAPHIC NOTES

The research literature has described several testing methods that are generally based on the idea that the input space of the test object should be divided into subsets, with the assumption that all inputs in the same subset cause similar behavior. These are collectively called *partition testing* and include equivalence partitioning [Myers, 1979], boundary value analysis [Myers, 1979], category partition [Ostrand and Balcer, 1988], and domain testing [Beizer, 1990]. An extensive survey with examples was published by Grindal et al. [Grindal et al., 2005].

The derivation of partitions and values started with Balcer, Hasling and Ostrand's category partition method in 1988 [Balcer et al., 1989, Ostrand and Balcer, 1988]. An alternate visualization is that of classification trees introduced by Grochtman, Grimm and Wegener in 1993 [Grochtmann et al., 1993, Grochtmann and Grimm, 1993]. Classification trees organize the input space partitioning information into a tree structure. The first level nodes are the parameters and environment variables (characteristics); they may be recursively broken into sub-categories. Blocks appear as leaves in the tree and combinations are chosen by selecting among the leaves.

Chen et al. empirically identified common mistakes that testers made during input parameter modeling [Chen et al., 2004]. Many of the concepts on input domain modeling in this chapter come from Grindal's PhD work [Grindal, 2007, Grindal and Offutt, 2007, Grindal et al., 2007]. Both Cohen et al. [Cohen et al., 1997] and Yin et al. [Yin et al., 1997] suggest functionality-oriented approaches to input parameter modeling. Functionality-oriented input parameter modeling was also implicitly used by Grindal et al. [Grindal et al., 2006]. Two other IDM-related methods are Classification Trees [Grochtmann and Grimm, 1993] and a UML

activity diagram -based method [Chen et al., 2005]. Beizer [Beizer, 1990], Malaiya [Malaiya, 1995], and Chen et al. [Chen et al., 2004] also address the problem of characteristic selection.

Grindal published an analytical and empirical comparison of different constraint handling mechanisms [Grindal et al., 2007].

Stocks and Carrington [Stocks and Carrington, 1996] provided a formal notion of specification-based testing that encompasses most approaches to input space partition testing. In particular, they addressed the problem of refining test frames (which we simply and informally call test requirements in this book) to test cases.

The each choice and base choice criteria were introduced by Ammann and Offutt in 1994 [Ammann and Offutt, 1994]. The extension to multiple base choice was alluded to in their 1994 paper ("*Many systems have multiple normal modes of operation, but we consider only one normal mode for simplicity here.*"), but was first defined in this book. Cohen et al. [Cohen et al., 1997] indicated that valid and invalid parameter values should be treated differently with respect to coverage. This allows the base choice criterion to implement a type of stress testing. *Valid* values lie within the bounds of normal operation of the test object, and *invalid* values lie outside the normal operating range. Invalid values often result in an error message and the execution terminates. To avoid one invalid value masking another, Cohen et al. suggested that only one invalid value should be included in each test case.

Burroughs et al. [Burroughs et al., 1994] and Cohen et al. [Cohen et al., 1997, Cohen et al., 1996, Cohen et al., 1994] suggested the heuristic Pair-Wise Coverage as part of the Automatic Efficient Test Generator (AETG). AETG also includes a variation on the base choice combination criterion. In AETG's version, called *default testing*, the tester varies the values of one characteristic at a time while the other characteristics contain *some* default value. The term "default testing" was also used by Burr and Young [Burr and Young, 1998], who described yet another variation of the base choices. In their version, all characteristics except one contain the default value, and the remaining characteristics contain a maximum or a minimum value. This variant will not necessarily satisfy Each Choice Coverage.

The Constrained Array Test System (CATS) tool for generating test cases was described by Sherwood [Sherwood, 1994] to satisfy pair-wise coverage. For programs with two or more characteristics, the in-parameter-order (IPO) combination strategy [Lei and Tai, 2001, Lei and Tai, 1998,

Tai and Lei, 2002] generates a test set that satisfies pair-wise coverage for the first two parameters (characteristic in our terminology). The test set is then extended to satisfy pair-wise coverage for the first three parameters, and continues for each additional parameter until all parameters are included.

Williams and Probert invented *t*-wise coverage [Williams and Probert, 2001]. A special case of *t*-wise coverage called *variable strength* was proposed by Cohen, Gibbons, Mugridge, and Colburn [Cohen et al., 2003]. This strategy requires higher coverage among a subset of characteristics and lower coverage across the others. Assume for example a test problem with four parameters *A, B, C, D*. Variable strength may require 3-wise coverage for parameters *B, C, D* and 2-wise coverage for parameter *A*. Cohen, Gibbons, Mugridge, and Colburn [Cohen et al., 2003] suggested using Simulated Annealing (SA) to generate test sets for *t*-wise coverage. Shiba, Tsuchiya, and Kikuno [Shiba et al., 2004] proposed using a genetic algorithm (GA) to satisfy pair-wise coverage. The same paper also suggested using the ant colony algorithm (ACA).

Mandl suggested using orthogonal arrays to generate values for T-Wise Coverage [Mandl, 1985]. This idea was further developed by Williams and Probert [Williams and Probert, 1996]. Covering Arrays [Williams, 2000] is an extension of orthogonal arrays. A property of orthogonal arrays is that they are *balanced*, which means that each characteristic value occurs the same number of times in the test set. If only *t*-wise (for instance, pair-wise) coverageis desired, the balance property is unnecessary and will make the algorithm less efficient. In a covering array that satisfies *t*-wise coverage, each *t*-tuple occurs at least once but not necessarily the same number of times. Another problem with orthogonal arrays is that for some problem sizes we do not have enough orthogonal arrays to represent the entire problem. This problem is also avoided by using covering arrays.

Several papers have provided experiential and experimental results of using input space partitioning. Heller [Heller, 1995] used a realistic example to show that testing all combinations of characteristic values is infeasible in practice. Heller concluded that we need to identify a subset of combinations of manageable size.

Kuhn and Reilly [Kuhn and Reilly, 2002] investigated 365 error reports from two large real-life projects and discovered that pair-wise coverage was nearly as effective at finding faults as testing all combinations. More supporting data were given by Kuhn and Wallace [Kuhn et al., 2004].

Piwowarski, Ohba, and Caruso [Piwowarski et al., 1993] described how to apply code coverage successfully as a stopping criterion during functional testing. The authors formulated functional testing as the problem of selecting test cases from all combinations of values of the input parameters. Burr and Young [Burrand Young, 1998] show that continually monitoring code coverage helps improve the input domain model. Initial experiments showed that ad hoc testing resulted in about 50% decision coverage, but by continually applying code coverage and refining the input domain models, decision coverage was increased to 84%.

Plenty of examples of applying input space partitioning in practice have been published. Cohen, Dalal, Kajla and Patton [Cohen et al., 1996] demonstrated the use of AETG for screen testing, by testing the input fields for consistency and validity across a number of screens. Dalal, Jain, Karunanithi, Leaton, Lott, Patton and Horowitz [Dalal et al., 1999, Dalal et al., 1998] report results from using the AETG tool. It was used to generate test cases for Bellcore's Intelligent Service Control Point, a rule-based system used to assign work requests to technicians, and a GUI window in a large application. Offutt and Alluri built a special purpose input space partitioning tool for Freddie Mac [Offutt and Alluri, 2014], and found that the technique not only increased the number of faults detected during testing, but also significantly reduced the cost of testing.

Burr and Young [Burr and Young, 1998] also used the AETG tool to test a Nortel application that convertsemail messages from one format to another. Huller [Huller, 2000] used an IPO -related algorithm to test ground systems for satellite communications.

Williams and Probert [Williams and Probert, 1996] demonstrated how input space partitioning can be used to organize configuration testing. Yilmaz, Cohen and Porter [Yilmaz et al., 2004] used covering arrays as a starting point for fault localization in complex configuration spaces.

Huller [Huller, 2000] showed that pair-wise configuration testing can save more than 60% in both cost and time compared to quasi-exhaustive testing. Brownlie, Prowse, and Phadke [Brownlie et al., 1992] compared the results of using Orthogonal Arrays (OA) on one version of a PMX/StarMAIL release with the results from conventional testing on a prior release. The authors estimated that 22% more faults would have been found if OA had been used on the first version.

Several studies have compared the number of tests generated. The number of tests varies when using non-deterministic algorithms. Several

papers compared input space partitioning strategies that satisfy 2-wise or 3-wise coverage: IPO and AETG [Lei and Tai, 2001], OA and AETG [Grindal et al., 2006], Covering Arrays (CA) and IPO [Williams, 2000], and AETG, IPO, SA, GA, and ACA [Shiba et al., 2004, Cohen et al., 2003]. Most of them found very little difference.

Another way to compare algorithms is with respect to the execution time. Lei and Tai [Lei and Tai, 1998] show that the time complexity of IPO is superior to that of AETG. Williams [Williams, 2000] reported that CA outperforms IPO by almost three orders of magnitude for the largest test problems in his study.

Grindal et al. [Grindal et al., 2006] compared algorithms by the number of faults found. They found that BCC performed as well as AETG and OA despite fewer test cases.

Input space partitioning strategies can also be compared based on their code coverage. Cohen etal. [Cohen et al., 1994] found that test sets generated by AETG for 2-wise coverage reach over 90% block coverage. Burr and Young [Burr and Young, 1998] got similar results for AETG, getting 93% block coverage with 47 test cases, compared to 85% block coverage for a restricted version of BCC using 72 test cases.

---

[1] We choose to use the term “blocks” to refer to the pieces of a partitioning. The term “partition” is often used for both conceptsin the research literature.

[2] As of this writing, the Iterator interface is online at: docs.oracle.com/javase/7/docs/api/java/util/Iterator.html

[3] Whether strong preconditions are a good idea at all is a contentious issue well beyond the scope of this text.